Crema Italia — Store Operating Standards

Version 1.18 · 2026-08-29 Source of truth: this file (`docs/standards/store-operating-standards.md`) in the theme repo. **Companion standards:** Brand Standards v2.3 (look & voice) · Collaboration Standard v1.1 (how we work).

EVERY VALUE IN THIS STANDARD IS PROVISIONAL UNTIL THE SITE GOES PUBLIC (Steve, 2026-08-24)

While we are in the POC process and before the site is live, treat every number here as a modelling placeholder, not a settled decision. Prices, costs, markups, discounts, thresholds, minimums, maximums, windows and cadences are all in that category. They are good enough to build and reason against. They are **not** good enough to charge money against, and none of them acquires authority merely by having been written down, versioned, and rendered to a PDF.

There is one deliberate analysis pass, and it happens at a specific moment: after the live site is built, before it is made public. Not earlier - the numbers cannot be validated against a store that does not exist. Not later - once we are public the values are commitments customers act on. **After that point we live with our wins and our losses, good decisions and poor ones alike.** Before it, changing a number costs nothing but the edit.

What this does and does not license. It does **not** license leaving contradictions in place: a value that disagrees with another value is a defect at any stage, because we are reasoning against these numbers now. It **does** mean that "we already decided that" is not an argument against re-examining a number before launch. The decision log records what we chose and why; it does not make the choice correct.

Do not preserve superseded pre-launch values as history. Nobody transacted under them, so they are drafts rather than facts. What becomes worth keeping is the first value we actually go live with, and what it then does in the market - that is a real data point for the next change. The version changelog below carries what moved between revisions; the body states only what is true.

Tracked as §12.13, which cannot close before launch.

v1.18 (2026-08-29) records the SKU decisions Steve locked that day. **No rule changed for any existing product and nothing is repriced.** The SKU format is promoted out of a subsection of the build spec into `docs/standards/sku-standard.md` - a normative repo document, deliberately **not** a fourth Standard: no version, no render, no badge, cited by section title. It moved because it had lived in exactly one document since 2026-08-21 and could not be found by the person who decided it; **the coordinator detects contradiction between documents, and a decision recorded in one document contradicts nothing.**

§7 gains three things. The BOM is stated as five classes of line - the collection, N coffees, one card, one box - with **lot control, roast-date capture and the freshness windows applying to coffee alone**, which is what a receiving procedure needs. The BOM is **transmitted per order**; the 3PL is not asked to hold our recipe. And a **pre-enable gate** is added: a collection is not enabled until its coffees, card and box are all physically at the 3PL - the component gate is continuous and automatic, this one is a person, once.

One latent contradiction closed in the same pass. §2.2 and §2.3 price an aged collection, and nothing could produce one: the availability gate read the fresh window unconditionally, so an Offerta collection was unpublishable by construction. §7 now reads the band appropriate to the collection's shelf, and states that an aged collection is a **deliberately curated new collection**, never an automatic transition - which makes that factor live rather than vestigial.

§12.9(0) is sharpened from three open candidates to one chosen answer (one bin, two pick conditions; marked-bin fallback; per-unit transfer cost quoted before selection), and **§5.5's lint gains two SKU rules** - while saying plainly that the lint still does not exist, and that it could never catch a SKU hand-typed into the Shopify admin.

v1.17 (2026-08-24) corrects a claim v1.16 introduced hours earlier, and replaces it with the arithmetic that should have been run first. v1.16's §12.3 said outbound shipping to the customer *"appears nowhere in the pricing formula"* and that the matrix *"omits a cost line that scales with volume."* **The first half is true of §2.2 and the second is wrong.** The landed-cost model carries outbound shipping in full - Inputs §D, into per-order variable cost, into the MSRP-at-target-margin sheet. Nothing was omitted.

What is actually wrong is larger, and it is now computed rather than asserted: §2.2's markup matrix and the cost model have never been reconciled, and at the model's own most-likely inputs they disagree severely - three of five representative SKUs price below total unit cost. §12.3 now carries the table. Two stale model inputs are recorded with it. **No rule changed and nothing is repriced**; this is §12.3 becoming specific.

v1.16 (2026-08-24) lands two decisions Steve locked on 2026-08-23, and adds the provisional-values rule above (§12.13) and shipping-rate governance (§8.3).

Outbound shipping repriced (§8): free at **$69+**, flat **$12.50** under, free tier **contiguous U.S. only**, threshold measured **after discounts**. Roccia subscription shipping unchanged - free, no minimum. §8.3 is new and sets when the rates are reviewed, which lever moves first, and why the exposure is weight and zone rather than dollars.

Freshness values become dated declarations (§5.5): no live freshness rule states a number in prose, it names the token. Six sites corrected; `days_to_offerta` removed from every live surface. §5.5 carries the classification rule - which numbers become tokens, and which must stay literals. Storage moves off theme settings, because `settings_data.json` does not survive a theme swap and this store spins up preview themes routinely; **that migration is not built yet and §5.4 says so.** No window changed.

§12.3 amended: outbound shipping to the customer is absent from the pricing formula entirely, which matters because we absorb it on most orders.

v1.15 (2026-08-22) changes **which system owns the subscriber rate**, on measured platform behaviour rather than intent. §3's no-codes `MAX` policy is untouched. The v1.3 mechanism - one Shopify Function evaluating every discount - was built and tested against Loop on a dev store and fails twice: a Function discount **compounds** with a selling-plan price adjustment (measured at an effective 20.76% where 12% was intended, and `combinesWith: false` does not prevent it, because a plan adjustment is not a discount), and **Functions are not re-run on recurring orders** - the rate is snapshotted onto the contract at signup. So the **Loop selling plan now owns subscription lines** and the **Function owns one-time lines**, with `appliesOnSubscription: false` as the guard. One consequence is accepted knowingly: the **win-back 15%** is the only campaign that could out-rank the standing rate on a subscription, and under this model it cannot; a designed-but-unbuilt **top-up to `MAX`** is the remedy if that ever matters. §12.7 closes **verified yes**, §12.8 closes **answered**. Nothing is repriced and no rate changed.

v1.14 (2026-08-21) retires **`peak_flavor_days`** as a setting and closes §12.10, which v1.13 had opened hours earlier. The confusion was never the number - *"brew within 30 days"* sat inches from Offerta's *"Best within 27 days"* and the two measured from different things. The other windows are **gates we enforce**; this was **advice about behaviour we do not control**, so it is brand copy rather than tunable policy. It is now merged into the whole-bean sentence, counting **from receiving** rather than from roast, so one statement carries the purchase expectation, the freshness advice and the route to a grinder. No window changed and nothing is repriced.

v1.13 (2026-08-21) changes both freshness statements and **reverses one v1.12 rule**. Offerta no longer shows an actual roast date; it shows a **computed band** - *"Roasted between 24-MAR-2026 and 22-MAY-2026"* - bounded by the donate age at one end and by one day older than the main-shelf floor at the other. v1.12 assumed an Offerta product is a single split-off lot; **a slow-moving SKU can accumulate more than one**, so a single date was only mostly honest. The main-shelf sentence becomes *"These beans are within our best-freshness window of N days"*, dropping the peak-flavour clause, which read ambiguously after a roast-date statement. That leaves `peak_flavor_days` without a consumer, so **§12.10 is opened** to decide where the message goes rather than letting a setting sit unread. Nothing is repriced, and no window changed.

v1.12 (2026-08-21) resets the freshness windows and replaces the display rule. **Main shelves move from 60 to 90 days; Offerta is 91-150; past 150 the coffee comes off sale and is donated.** `peak_flavor_days` stays at 30 and is clarified as a **brewing message to the customer**, not a shelf-life rule. A new governance rule: **windows may be shortened, never extended** - the 60-to-90 move is a one-time recalibration made while pre-launch, when nobody has bought under the old promise. The display becomes a **computed floor** - *"Roasted on or after 23-MAY-2026"*, today minus the window, server-side - replacing v1.11's roast-date range, which depended on lot data being entered on time and whose fresh end was unreachable under FIFO anyway. Offerta keeps its actual roast date. **`DD-MMM-YYYY` is now required wherever a date is shown to anyone.** A third qualifying question is added to §12.9: how a 3PL segregates an Offerta lot from fresh stock of the same SKU. Nothing is repriced.

v1.11 (2026-08-20) adds **§5.4 Fulfilment order and multiple lots** and **§6.1 Substitution on a subscription**, neither of which existed anywhere. Both came out of Steve spotting that the POC models a case that never happens - one lot per coffee - while the real business routinely holds several. We ship **FIFO by roast date, per shelf**, on the sellable pool only; the customer sees a **roast date range** rather than a single date, and the **best-by date is no longer displayed** because it is the roast date plus a constant. A subscription is never filled with coffee past the freshness promise on its ship date, which is the substitution trigger. Substitution never crosses caffeine and never costs the customer more. Three questions about *how* a substitution is offered are **left explicitly open** in §6.1 rather than answered by assumption. Nothing is repriced.

v1.10 (2026-08-20) closes the one item v1.9 left open, as **§13.5.2**: **Bottega is rated, and is its own rating context.** The comparison objection that keeps a global average suspect is an argument about *coffee*, where the palate is the variable; it does not apply to equipment, where a grinder either holds its setting or does not. Bottega is already an exception the customer is told about, so this is consistent rather than special-cased. Two binding consequences: it **never shows a reorder rate** (nobody rebuys a grinder, and the figure is excluded by shelf rather than left for a sample floor to eventually pass), and it **never gets the palate-matched layer**. It also becomes the one shelf where a card-level rating could later be defended on its own terms. No rule elsewhere changes.

v1.9 (2026-08-20) adds **§13.5.1**, governing *where* the rating control may appear. It may render **only on the detail view of a purchasable product** - never on a roaster profile, person page or editorial surface - and it is **kept off product cards in grids** below a catalogue coverage floor, because a grid is a comparison device and this rating is explicitly not comparable, because a wall of "Not yet rated" advertises an empty store, and because uneven coverage makes unrated coffees read as ignored. Above the floor, only rated cards may show the mark; **a null is never rendered in a grid.** Note the detail view is the opposite: it renders the control *including* when empty, which is what prompts a purchaser to rate at all. Leaves one item open by design - whether Bottega equipment is exempt from the comparison objection. No rule elsewhere changes.

v1.8 (2026-08-20) adds **§13 Reviews & social proof**, closing the last open item on the storefront scorecard. Reviews are **purchase-gated only** (emailed per-order link; the public form is disabled - an account is free, so a login gate proves nothing and the order is the trust anchor); reviewers may choose a display name while the customer identity is retained as the join key to their taste profile; **everything but abusive content is published**, with *abusive* defined so it is not re-decided daily; **no photograph reviews**; the rating renders through our own discreet control even at zero, with approved empty-state copy; `aggregateRating` is emitted only when a real review exists; and reorder rate and palate-matched feedback carry **minimum-n floors with silence below them**. Numbered 13 deliberately - renumbering §10-§12 would falsify citations in immutable §9 log entries. Nothing is repriced.

v1.7 (2026-08-20) is an **editorial repair, no rule change and no renumbering.** §12.9 (the 3PL packing-slip and insert question) had been appended *after* the document's own closing citation block, so it sat outside the document it belongs to - readers of the render met the "end of Standard" stamp and then more Standard. It is now item 9 of the §12 numbered list, in the same shape as items 1-8, and the citation closes the file as it should. The text of the item is unchanged. Bumped rather than silently re-rendered because a same-version content edit is the one drift no version-stamp check can see (see the `f9ffcb1` incident, 2026-08-04).

v1.6 (2026-08-19) removes the §12.9 opened hours earlier by v1.5, which asked who would hold the Founding Member slot if a gift subscription were sold. Steve: *"we've already said there is no gift subscription capability... then why do you ask about Founding Member slot?"* Correct — §8.2 says subscriptions cannot be gifted, and that **is** the decision. Parking entitlement rules for a product we have declined to build put speculative scope into a list whose purpose is holding items that must close **before the production build**; nothing depends on it. §8.2 now states the rule without deferring anything, and the 3PL item renumbers to §12.9. No other change.

v1.5 (2026-08-19) adds **§8.1 — nothing inside a package shows a price**, gift or not, with the receipt as an **email entitlement.** Steve's call, and deliberately blanket rather than a gift-order exception: a conditional rule has to be executed correctly on every order by whoever is packing, and it fails silently. It also adds **§8.2 Gifting** (order-level only, never inferred from a differing shipping address, subscriptions excluded). It also opened a §12.9 on the Founding Member slot under a gift subscription, **which v1.6 removed as speculative** — the §12.9 in this document today is the 3PL item. Nothing is repriced.

v1.4 (2026-08-19) is a **vocabulary correction, no rule change.** "Tour" was being used throughout as the *name of the archetype* — "Tour / bundle pricing", "Tours / bundles — the BOM model", "Sorpresa ships only as Tours". It is not the archetype; it is a **SKU name**. Steve: *"Tour is simply an SKU name (description) and is NOT a website term."* The archetype is a **collection**, and a collection need not be a tour of anywhere — `Decaf Collection 1` and `Roaster's Favorites 2` are the same thing as `Tour d'Italia 1`. Every archetype usage in this Standard is now "collection"; product names are unaffected. The same sweep ran across the storefront copy and `docs/production_build_spec.md`, and the rule is recorded in `CLAUDE.md` §6. See §1.1 below. Rules, pricing factors and matrices are **unchanged**.

v1.3 (2026-07-25) replaced the unachievable **"No visible promo-code field at checkout"** exclusion (§10) with the rule we can actually hold: **we issue no discount codes at all.** Checkout's code field is visible and functional on every plan below Shopify Plus — it cannot be hidden, disabled, or made read-only without Plus (~$24k/yr over Advanced for that one field; **Steve declined**). It is instead **permanently inert in practice, because no valid code exists anywhere.** Every discount — standing and campaign alike — is computed **server-side** and applies automatically. This **supersedes v1.2's "campaign discounts apply via URL parameter or personalized email link"** (§3): a `/discount/CODE` link *is* a real Shopify code, readable straight out of the URL and postable to a coupon site, which is the leak this closes. **Policy is locked; the mechanism (§11) is the intended implementation and is NOT yet verified against the platform** — see §12.7 and §12.8.

v1.2 (2026-07-13) replaced additive discount stacking with a **no-stacking, highest-wins (`MAX`)** rule (§3): a customer receives only the single highest discount they qualify for — a founder who is also a first-time buyer gets **12%** (not 17%); a first-time buyer who earns the 3-bag volume tier gets **10%** (not 15%). The `MAX` rule *is* the cap, so there is no separate discount ceiling. Split the delivery mechanism (standing benefit auto-applied server-side vs campaign discounts via link). **BFCM** is no longer additive — it is a flat candidate in the `MAX`. Marked the **Referral** reward + capture **TBD** (§3, §12.6) since the former "free 100g bag" has no standalone SKU. Annotated the vestigial **Sorpresa 250g / O250g** matrix cells as retired (§2.2). **Note:** the POC discount code still stacks and is now knowingly behind this Standard — a POC/production fix, tracked separately.

v1.1 (2026-07-13) closed the three open decisions from v1.0 §12: per-SKU markup override added (§2.2), price-maintenance tool approach set (§11), and the subscriber-benefit / pause-cancel model settled (§3.1) — including making **Founding Member status durable** (account-level, lost only on account closure), which supersedes the 2026-07-10 Active/Forfeited model (§4).

What this document is. The canonical, human-and-machine-readable statement of **how the Crema Italia store buys, prices, sells, and fulfils** — the commerce mechanics. Brand Standards owns how the store *looks and speaks*; this owns how it *does business*. It is a **Standard** (what is true *now*), not a log (what changed, when) — history lives in `CLAUDE.md` §9 and `Coordination/DECISIONS_LOG.md`.

How to use it. When a commerce rule changes, update THIS file in the same pass and bump the version, then log the event. Other documents (the POC, code comments, `CLAUDE.md`) should *point at* this file, never restate its rules — restating is what causes drift. The POC is an ephemeral mock we build *from*; this document is what we build *to*, and what future alterations are measured against.

Provenance. v1.0 consolidates the rules previously locked across the (now-retired) Shopify Magic Build Prompt v3 FINAL, the `01_Metafield_Reference` doc, and the locked decisions in `CLAUDE.md` §9 (esp. 2026-06-29, 2026-07-04, 2026-07-10). Where those sources conflicted, the conflict is called out in **§12 Open Decisions** rather than silently resolved.

---

1. The shelves — canonical definitions

Every product lives on exactly one shelf. The four **coffee** shelves drive navigation, collections, and taxonomy; **Bottega** is a separate non-coffee retail section, not a fifth coffee shelf.

Shelf	English meaning	Commercial role	Key rules
Roccia	"Rock / staple"	The subscription backbone — everyday coffees sold one-time **and** as Roccia subscriptions	Bag sizes 250g / 500g / 1kg (**no 100g**). The only shelf with subscriptions.
Sorpresa	"Surprise"	Discovery **collections** (composite BOM SKUs)	Sorpresa 100g bags exist **only inside collections**, never sold alone. One-time only.
Selezione	"Selection"	Premium / seasonal / limited micro-lots	One-time purchase only, never a subscription. Honest scarcity ("low inventory" under 12 units, hard cap). Active Roccia subscribers see new SKUs 48h early (Locksmith, tag `active-roccia`).
Offerta	"Offer / deal"	Aged lots moved to an honest markdown as they approach their freshness limit	Not stocked directly — items *transition* here by age (see §6). Uses the `O[size]` markup factors. Guarantee is "as-is, defects only" (see §5).
Bottega	"Shop / workshop"	Non-coffee retail: equipment, accessories, Crema Italia merch	Independent ordering experience. No roaster affiliation, no freshness/expiry logic. Never appears on roaster profiles. Never discounted for subscribers/Founding.

1.1 Vocabulary — *collection* is the term, *Tour* is a name

A Sorpresa product is a **collection**: a named, curated set of component coffees sold as one composite BOM SKU (§7).

"Tour" is a SKU name, not a category term (Steve, 2026-08-19). `Tour d'Italia 1` and `Tour Tuscany` are *names of collections*, in the same way `Decaf Collection 1` and `Roaster's Favorites 2` are — all three are the same archetype, and only the first two happen to be tours of anywhere. The word entered the vocabulary because the first worked example was regional; it then spread across the storefront, this Standard and the build spec as though it were the category itself, which quietly narrowed the shelf to one kind of product.

Applies to every surface: storefront copy, this Standard, `docs/production_build_spec.md`, and future admin/ops language. Name a product whatever it should be called; describe the archetype as a **collection**. Recorded as a "Never" in `CLAUDE.md` §6.

---

2. Pricing model

2.1 The formula (LOCKED)

```
Retail price = SKU_LAST_COST × Markup[Shelf, Size]
```

- **SKU_LAST_COST** = the US-dollar **landed cost of the most recent lot** purchased for that SKU (EUR purchase price + freight + tariff + handling), **locked at receipt** on the lot's variant (`landed_cost_usd`).
- **Markup** comes from the fixed matrix below, keyed by shelf and bag size.
- The retired `EUR × 0.60 × markup × 1.165` formula is **superseded — do not use.**

2.2 Markup matrix (LOCKED; the *table* is admin-configurable)

`O[size]` columns = the Offerta (aged-lot) variant of that size.

Shelf	100g	250g	500g	1kg	O100g	O250g	O500g	O1kg
Roccia	—	2.8×	2.5×	2.2×	—	2.3×	1.7×	1.5×
Sorpresa	3.7×	—	—	—	3.2×	—	—	—
Selezione	3.7×	3.0×	2.7×	2.4×	3.2×	2.5×	2.2×	1.9×
Offerta	*uses the `O[size]` factor of the item's originating shelf*							
Bottega	2.0× flat (1.5× if clearing)							

> **Sorpresa 250g / O250g are intentionally blank (retired 2026-07-13).** Sorpresa ships **only as collections** (§2.3), priced off the 100g factor (3.7× fresh / 3.2× aged); there is no standalone Sorpresa 250g product, so those cells carry no live factor. If a standalone Sorpresa bag is ever introduced, restore the cells and log it.
>
> **Per-SKU override (LOCKED 2026-07-13).** The matrix is the **default** that governs every SKU by shelf/size — editing the table moves every SKU of that shelf/size together. In addition, a SKU may carry an optional `markup_override`: leave it blank (the normal case) and the SKU inherits the matrix; set it and it wins **for that SKU only**. That is the entire shape - one table-wide default plus an optional per-SKU override, and **no parallel field anywhere**. An override is a **deliberate exception, not the norm**, and it **routes through the same admin approval** as any price change (§2.4).

2.3 Collection / bundle pricing

- `Collection cost = Σ (component SKU_LAST_COSTs) + packaging_cost`
- `Collection retail = Collection cost × Sorpresa/100g factor (3.7×)`; an aged collection uses `3.2×`.
- Component cost changes flow into collection cost automatically; the customer-facing collection price updates **only on admin approval** (same governance as any SKU).
- *Worked example:* (6.00 + 5.50 + 7.00 components) + 2.50 packaging = 21.00 → ×3.7 = **$77.70.**

2.4 Price-update governance (LOCKED — no automatic passthrough)

A new lot never silently changes the shelf price. Instead: 1. New lot arrives → update `landed_cost_usd` → system computes the *proposed* new retail. 2. Admin is alerted ("current retail $X; new cost would yield $Y if approved"). 3. Admin **approves / holds (retain margin) / defers**. 4. Price changes only on explicit approval.

Rationale: avoid psychological pricing churn, protect margin on fast movers, keep pricing a deliberate decision.

---

3. Discounts & the subscriber privilege

No stacking — highest applicable discount wins (LOCKED 2026-07-13). A customer never receives two discounts at once. The applied rate is the `MAX` **of every discount they currently qualify for**; all others are obviated. This deliberately protects margin on thin per-bag markups: a founder who is also a first-time buyer gets **12%** (not 17%); a first-time buyer who earns the 3-bag volume tier gets **10%** (not 15%). There is **no additive stacking and no separate combined cap — the `MAX` rule is the cap.**

Delivery mechanism — POLICY (LOCKED 2026-07-25): we issue NO discount codes, of any kind, ever. Every discount in the table below — standing *and* campaign — is **computed server-side and applied automatically**. No customer is ever given a code, a coupon, or a `/discount/…` link, and none is ever published, promoted, printed, or referenced in any marketing.

Consequences, all deliberate:

- **The checkout code field stays visible.** It cannot be hidden, disabled, or made read-only below Shopify Plus, and Plus is not justifiable for it (§10). It is **inert in practice** — nothing a customer types can produce a discount, because no valid code exists to type.
- **Nothing can leak.** A `/discount/CODE` link carries a real, reusable Shopify code in plain sight; anyone can read it out of the URL and post it publicly. Issuing none removes that surface entirely. This is why v1.2's "URL parameter or personalized email link" wording is retired.
- **Campaign emails link to the store, not to a discount.** Eligibility already sits on the customer (a tag) or in the cart, so the offer applies on arrival with nothing to redeem.
- **One evaluator owns `MAX`.** Because every discount is computed in the same place, the single highest applicable rate is enforced by construction. Mixing codes with server-side discounts

would hand that decision to Shopify's own combination rules — exactly what the `MAX` rule forbids.

> **Policy above is locked. The *mechanism* that implements it lives in §11 and is NOT yet verified against the platform** (§12.7, §12.8). If the platform forces a change, §11 changes — this policy does not.

Discount	Rate	Applies to	Notes (every row competes in the ; none stack)
Founding Member	12%	Roccia, Sorpresa, Selezione	Auto when benefits on. Founder tier of the subscriber benefit — see §4.
Active subscriber	10%	Roccia, Sorpresa, Selezione	Auto when benefits on (regular tier).
Win-back	15%	All shelves	30 days post-Roccia-cancel; time-boxed customer tag, email links to the store. Wins over the standing benefit by design.
Volume	2 bags 5% / 3+ bags 10%	Coffee shelves except Offerta/Bottega	Competes in the `MAX`; for a subscriber the 10/12% already wins.
First-time buyer	5%	All shelves except Bottega	One-time, detected server-side (zero prior orders). A first-time founder/subscriber still gets 12/10% (higher).
BFCM	5%	All shelves incl. Offerta/Bottega	Manual admin toggle, site banner. A **flat** candidate in the `MAX` (no longer additive) — mainly benefits customers who hold no higher discount.
Abandoned cart	5%	All shelves	Triggered at email #3 as a time-boxed customer tag (the email links to the store, not to a code); once per customer per 90 days.
Referral	**TBD**	**TBD**	Referral capture is **not built yet**; reward + mechanism are open (§12.6). The former "free 100g bag" is void — 100g exists only inside collections (§1), so there is no standalone 100g SKU to gift. **Whatever form it takes, it must not require issuing a code** (the no-codes policy above).

3.1 Subscriber benefits & how long they last (LOCKED 2026-07-13)

"Subscriber benefits" = the discount (12% founder / 10% regular) *and* the shipping offsets (free/reduced shipping). They are bound to being a subscriber, with a deliberate win-back grace:

- **ON** while the account has **≥1 actively-shipping subscription.** A customer may hold several; any one shipping keeps benefits on. Pausing or cancelling *some* changes nothing while another ships.
- **60-day grace.** When the account reaches **no actively-shipping subscription** — whether by **pausing all** or **cancelling all** — benefits **continue for 60 days** from that date, then lapse. The grace is intentional: it is our window to re-contact the customer and win them back on the

benefits they are about to lose (FOMO), and it means no one is punished the instant they pause or cancel.

- **Reinstatement.** Resuming or re-subscribing at **any** later date restores benefits at the account's tier — **12% if still a Founding Member, 10% if regular** (§4).
- Benefits never apply to **Offerta or Bottega** (§3 table), regardless of tier.

> **Supersedes** both the older "$45+ shipment delivered per calendar month" active-definition and the interim "cancel = immediate benefit loss" rule: **cancel-all now carries the same 60-day grace as pause-all.** The only thing that ends benefits *without* a grace is closing the entire account (§4).

No sitewide percentage-off promotional sales — the subscriber benefit is the standing perk, not a recurring sale. **No Italian-holiday discounting** — holidays are Journal editorial only.

---

4. Founding Member mechanic (LOCKED 2026-07-13 — supersedes 2026-07-10)

- **"Founding Member · No. NNN"** is a **durable, account-level status** — one of **222** numbered slots, granted at launch signup, with a numbered certificate. It is tied to the **account**, not to any subscription.
- **It survives cancelling any or all subscriptions.** A founder who cancels everything and re-subscribes months or years later returns a **Founding Member at 12%.** Founder status is not something a subscription action can take away.
- **The 12% is the *founder tier* of the subscriber benefit** (§3.1): a founder receives **12%** whenever subscriber benefits are ON (an actively-shipping subscription, or within the 60-day grace); a regular subscriber receives **10%** under the same conditions; benefits OFF → 0%.
- **Founding status is lost ONLY by closing the entire account** — *"Cancel the entire account,"* a distinct action from cancelling a subscription (e.g. a founder dies and a family member closes the account). On account closure the numbered founding **slot is released/retired.**
- **Retired (do not build):** the 2026-07-10 "Active/Forfeited" two-state model, the permanent 12%→10% forfeiture on a subscription cancel, and the one-way `founding_rate_forfeited` tag. **There is no permanent forfeiture** — the only terminal event for a founder is account closure.
- *Rationale:* the 12-vs-10 delta is tiny (~$10/yr) — a pride good. Making founder status durable and always-reinstatable removes any reason a founder would fear pausing or cancelling, and keeps the win-back door open.

---

5. Freshness & the Offerta transition

- **Freshness statement, main shelves (revised v1.13):** "These beans are within our best-freshness window of `{freshness_window_days}` days." The number is **resolved from the declaration, never**

typed (§5.4, §5.5, build spec §11). Never "best by" alone. **Offerta has its own statement** - see §5.4. The peak-flavour clause was removed from this sentence, and `peak_flavor_days` **was retired as a setting entirely** (v1.14). The confusion was never the number: *"brew within 30 days"* sat inches from Offerta's *"Best within 27 days"*, and the two measured from different things. Steve's diagnosis: the other three windows are **gates we enforce and control**, whereas this one is **advice about behaviour we do not control** - how a customer chooses to consume what they bought. Nothing branches on it, nothing enforces it, and it never varies by SKU or shelf, so representing it as tunable policy invited someone to tune a piece of advice. It is **brand copy** and belongs in the repo where the voice rules apply, not in the settings that drive gating logic - a category-2 string under build spec §11, not a category-3 one.

The advice now lives in the whole-bean sentence, merged rather than adjacent (approved copy, Steve, 2026-08-21):

> Whole beans only. We recommend using your beans within 30 days of receiving them, and grinding them just before each brew. Need a grinder? Search for one in our Bottega.

One statement doing three jobs - the purchase expectation, the freshness advice, and the route to a grinder - instead of two sentences saying overlapping things a few lines apart. Note the 30 days counts **from receiving**, not from roast, which is what removes the ambiguity. **It is NOT paired with an actual roast date on the main shelves** - v1.12 replaced that with a computed floor, *"Roasted on or after DD-MMM-YYYY"*; only Offerta shows a real date (§5.4). - `days_to_offerta` **is RETIRED (v1.12).** It carried its own default of 45, which is the same fact as the freshness window stated a second time - and by 2026-08-21 the two had diverged, 45 against 90. Coffee moves to Offerta when it leaves the freshness window, so `freshness_window_days` **is the only home for that boundary.** A per-SKU override, if it is ever genuinely wanted, overrides that setting; it does not get a parallel field. - `offerta_transition_date` = `roast_date + {freshness_window_days}`. Coffee moves to Offerta when it leaves the freshness window; there is no second boundary. - **Nightly** the system flags lots at/over their transition date. Admin sees a daily digest and approves (may auto-approve). On transition: `current_shelf` → Offerta (Shopify Flow), price recalculates to the `O[size]` factor, 3PL gets a priority-ship (FIFO) flag. - **Offerta listing shows:** original price (struck through) + Offerta price + savings + the *actual* remaining freshness window (e.g. "best within 23 days"). - **Donation threshold:** coffee older than `{offerta_fresh_days}` is removed from sale and donated to **Feeding Tampa Bay** (the no-waste pledge). **The value is named, never typed** (§5.5). - **Offerta guarantee is modified:** "as-is, defects only" — the standard first-bag satisfaction guarantee (§9) does not apply to already-discounted aged lots.

5.4 Fulfilment order and multiple lots (Steve, 2026-08-20)

We routinely hold **more than one lot of the same coffee**, because we replenish while previous stock is still selling. Stock is fulfilled **first in, first out by roast date**, and **per shelf**: Roccia ships the oldest fresh lot, Offerta the oldest aged lot.

FIFO operates only on the **sellable pool**. A lot past the freshness window is off sale entirely (§5), so FIFO can never mean shipping something this Standard has already withdrawn.

The windows (values set 2026-08-21; this is the declaration site, and they are never typed into any page):

Field	Days	Meaning
`freshness_window_days`	**90**	Age 0-90: the coffee sits on Roccia, Sorpresa or Selezione
`offerta_fresh_days`	**150**	Age **91-150**: Offerta, priced accordingly. Past 150 it comes off sale entirely and is donated

This declaration is effective from 2026-08-21, and this table is the only place either number appears as a figure. Everywhere else - in this Standard, in the theme, in any generated document - the value is **named as a token and resolved at read time** (§5.5).

Boundaries are exclusive - 90 belongs to the main shelves and Offerta starts at 91. No day lives in two bands. **The Offerta lower bound is derived** (`freshness_window_days + 1`) and never stored as a third field. Storing it is exactly how `days_to_offerta` happened.

> **Where these live is CHANGING (decided 2026-08-23; NOT yet built).** They are held as **theme settings** today, and that is the defect: `settings_data.json` does not survive a theme swap, and this store spins up and discards preview themes routinely, so a policy boundary can be silently reset by a theme replacement. They move to a store-level **`freshness_policy` metaobject** carrying its own `effective_from`, `revision` and `standard_version` - definition in §5.5, open questions in §12.12. **Until that migration ships the theme setting remains the operative store of record.** This note exists so nobody reads the intent as the state.

Windows may be SHORTENED but never extended. Lengthening walks back a promise customers already bought under. The 2026-08-21 move from 60 to 90 is a **one-time recalibration against research**, made deliberately while the store is pre-launch and **nobody has ever purchased under the 60-day promise**, so there is nothing to retract. The rule starts from here: as real consumption data arrives we tighten, which widens the gap against sellers who claim two years of sealed freshness.

Coffee only. Equipment and merchandise carry no roast date and do not age.

5.5 Freshness values are dated declarations, and which numbers are tokens (Steve, 2026-08-23)

Every freshness value is a dated declaration, and no live freshness rule states a number in prose - it names the token. Two parts, and the second is the one that prevents recurrence.

Why the rule exists, stated once here rather than annotated at every site it touches: §5.4 had been applying this pattern correctly to one claim and not the others, and the others drifted - §5's donation threshold read **60 days** while §5.4's own table read past **150**, a ninety-day contradiction inside one document that survived two version bumps. The pattern was right; it was applied once.

1. Storage is store-level, not theme-level. A `freshness_policy` metaobject, not `settings_data.json`. It survives theme replacement, is editable in Admin without touching theme code, and is readable from Liquid and the Admin API.

```
type: freshness_policy                  # crema_italia.freshness_policy (singleton)
  freshness_window_days     Integer     # age 0-N: main shelves
  offerta_fresh_days        Integer     # age N+1-M: Offerta; past M off sale and donated
  effective_from            Date        # when THIS declaration took effect
  revision                  Integer     # monotonic; bump on every value change
  standard_version          Text        # the Store Operating version that authorised this revision
  notes                     Text        # one line: what changed and why
```

2. Every declaration carries an effective date. *A tunable with no date is exactly as driftable as a literal - you simply cannot see the drift.* With one, a reader, a template or an agent can assert that the value it holds is the current declaration rather than assuming it: a consumer holding a `revision` older than the live one is stale **by construction, not by inspection**.

The classification rule - and the half that matters is what it leaves alone. Not every number in these sections should become a token. Templating the history would mean the record of what changed changes whenever policy changes.

Class	Treatment	Example
Tunable gate - logic branches on it, it can be retuned	**Declaration field. Named as a token in prose, never a literal.**	`freshness_window_days`, `offerta_fresh_days`
Historical narrative - a record of a change that happened	**Keep the literal.** History must not move when policy moves.	"the 2026-08-21 move from 60 to 90 is a one-time recalibration"
Brand copy - advice we do not enforce	**Keep the literal**, in the repo where the voice rules apply.	"use your beans within 30 days of receiving them" (retired as a setting in v1.14, deliberately)
External fact - a claim about the market, not about us	**Keep the literal**; keep the substantiation on file.	the 24-month commercial-coffee claim
Contractual term - a number a counterparty signed	**Keep the literal. Never tokenise.**	the roaster's 12-month sealed-bag stability warranty

Governance is unchanged, and now belongs to the admin surface rather than only to this document: windows may be **shortened, never extended** (§5.4). A save that increases either window should be **rejected with the reason**, because the rule exists precisely for the moment someone is tempted.

Two enforcement gates, so this stops relying on care. It relied on care and it failed.

1. **A no-bare-literal check.** Lint this source and the theme templates for bare integers adjacent to freshness vocabulary (*day, days, roast, fresh, donate, Offerta*) outside two allowlisted regions: the §5.4 declaration table and the version-history block. Fail the build with the offending line. **This is the check that would have caught the 60-versus-150 split on the day it was written.**

The same check carries two SKU rules (added 2026-08-29; SKU Standard, *Where SKUs are assigned*). A string matching the SKU shape is a **shelf-encoded or malformed SKU** if it is not exactly ten `A-Z0-9` characters, optionally followed by `-OFF`; and a **hard-coded SKU literal** if it appears in theme code or documentation outside the SKU Standard's own examples, since a SKU in a template is a fixture handle by another name (`docs/production_build_spec.md` §11, §12). **What this check cannot do is catch a hand-typed SKU**, because hand-entry happens in the Shopify admin and not in this repository. Do not let its presence imply otherwise: the guard against hand-entry is the generator plus validation at product onboarding (`docs/production_build_spec.md` §15.2).

The check itself is specified and NOT BUILT. There is no lint and no CI in this repository today. Recording the rules here is what makes them arrive with it when it is written; it is not a claim that anything is being enforced. 2. **Resolve-and-assert on read.** Every consumer reads `revision` and `effective_from` alongside the values.

> **Do not bulk-replace "60 days" in this document.** §3 carries an unrelated **60-day subscriber-benefit grace**, which is not freshness and not in scope. The two were separated deliberately in the theme for this exact reason - they share a value by coincidence, and a find-and-replace corrupts whichever one you were not thinking about.

A subscription is never filled with coffee that exceeds the freshness promise on its ship date. That is the trigger for §6.1.

What the customer sees (revised v1.12, Steve, 2026-08-21). An earlier draft showed a roast date **range** across lots in stock. That is replaced by something simpler and stronger.

Main shelves show a computed floor, not a fact about the bag:

> Roasted on or after 23-MAY-2026

where the date is **today minus `freshness_window_days`**, computed server-side. It is a **guarantee derived from policy** - *nothing we ship you is older than this* - and it is true by construction, because coffee past the window is off sale entirely (§5).

Three reasons it beats showing actual dates:

1. **It cannot go stale or lie.** It has no dependency on lot data being entered, entered on time, or entered correctly. An actual roast date would show the *previous* lot's date on coffee already shipping if a receipt were recorded late.
2. **A range's fresh end is unreachable.** Under FIFO a single-bag buyer always receives the oldest lot, so the upper bound of a range is systematically optimistic.
3. **It is comprehensible.** One date, one meaning, no arithmetic asked of the reader.

The best-by date is not displayed. It is the roast date plus the window, so showing both states one fact twice and aims the reader at a deadline rather than at freshness.

Offerta shows a computed BAND, not a date (revised v1.13, Steve, 2026-08-21).

> Roasted between `{today - offerta_fresh_days}` and `{today - (freshness_window_days + 1)}`

With today at 21-AUG-2026 and the windows at 150 and 90, that reads *"Roasted between 24-MAR-2026 and 22-MAY-2026"*. The band **is the Offerta definition made visible**: the older bound is the age at which we withdraw and donate; the younger bound is **one day older than the main-shelf floor**, so the two shelves can never claim overlapping freshness.

This reverses v1.12, which had Offerta showing its actual roast date on the reasoning that an Offerta product is one split-off lot. That assumption does not hold: **a slow-moving SKU can accumulate more than one lot on Offerta.** Steve: *"quoting the range keeps the website always honest, instead of mostly honest."*

It is the same reasoning that produced the main-shelf floor, one shelf over - state the **guaranteed band derived from policy** rather than a specific fact that may not be true of every bag in the bin - and it carries the same benefits: no dependency on lot data, nothing that can go stale, and nothing that a late receipt can make into a lie.

The original rationale survives and is better served. The two shelves must not *look* identically fresh. A band does that more plainly than an actual date, because it makes the age gap explicit instead of leaving the reader to compare two dates and work it out.

Offerta's freshness line is "Best if used soon after purchase - sold as-is." A computed remaining-days figure would be a third number on the same subject.

`roast_date` **is still required on an Offerta product** - for the withdrawal trigger at `offerta_fresh_days` and for operations. It simply stops driving anything the customer sees.

Date format is `DD-MMM-YYYY` (e.g. `29-AUG-2026`) wherever a date is shown to anyone, customer or partner. `03/07/2026` is 3 July to an Italian roaster and 7 March to a U.S. warehouse; that ambiguity would break FIFO picking and mislead a customer, and it will otherwise happen.

FIFO itself is explained in the FAQ, not on the product page. With a computed floor there is nothing on the product page that needs explaining.

Approved customer copy (Steve, 2026-08-20). Belongs in the FAQ, with the roast range and the FIFO line surfacing on the product page:

> We purchase the same coffee routinely which is why we sometimes quote a roast date range. We always fulfill orders in the order that we receive inventory, or First In First Out (FIFO), in inventory-speak. If a particular lot is out of our immediate freshness window, it is moved to our Offerta shelf and priced accordingly. We do not fulfill subscriptions with products that exceed our freshness promise on the date of shipping to you.
>
> In rare circumstances where an Offerta lot exceeds our freshness promise entirely, we endeavor to donate this coffee to worthy recipients. Remember, commercial coffee is typically sold as fresh within a 24-month freshness window.
>
> Please note that any order of more than one bag may span lots, but the freshness of spanned lots is always in the favor of the customer. We want you to enjoy fresh coffee.

Substantiation for the 24-month claim. It is an objective claim about the market and needs a reasonable basis held on file - not a footnote on the page, which would read defensive and off-register. The basis is in OneDrive `CremaItalia LLC\Brand and Marketing\Market Research\`, principally *Deep-dive competitive pricing research Aug 2026*, which records "dated roast/best-before windows (~24 months)" across the Italian brands surveyed and cites a competitor displaying a best-before of 05.2028 at point of sale. **Recorded here because nobody will remember in two years where it came**

from. The word "most" was dropped from an earlier draft: the sample is Italian competitor brands, so "most commercial coffee" claimed more than the evidence supports.

Two notes on the copy, both deliberate. An earlier draft said the move to Offerta was "automatic"; that was removed, because the *split* is human-triggered - someone decides how many units move - while only the end-of-window withdrawal is automated (`docs/production_build_spec.md` §13.9). And a comparative closing clause was cut: this brand states a fact and lets the reader draw the conclusion, rather than arguing against other sellers (Brand Standards §9).

This is the first rule that binds the warehouse rather than the storefront. It holds only if the 3PL actually picks that way, so it belongs in their SOP and should be spot-audited. A customer-facing promise resting on somebody else's habit is not a promise.

6. Roccia subscriptions (Loop)

- Engine = **Loop** (native `selling_plan_groups` + Shopify Checkout + Loop-hosted portal).
- **Cadences 4 / 6 / 8 weeks.** Customer selects roaster + SKU + size (**250g / 500g / 1kg only**) + cadence.
- Every Roccia shipment: **10% off + free shipping** (no minimum).
- **Self-service controls (no fee, no minimum commitment):**
- **Skip** one shipment; **swap** roaster/SKU/size up to 48h before order lock.
- **Pause** — a single subscription **or** all subscriptions — offered as a bounded window: **[next delivery cycle]** or **[next two delivery cycles]**, then auto-resumes. Pause is a short skip, **not** a long hold: anything longer than two cycles, the customer should **cancel** instead.
- **Cancel** — a single subscription **or** all subscriptions — stops shipping immediately.
- Reaching **no actively-shipping subscription** (by pause-all or cancel-all) starts the **60-day benefits grace** (§3.1); benefits reinstate on any resume/re-subscribe. Founder status is unaffected either way (§4).
- **Pallet-timing gap policy:** when a subscriber's SKU is between pallets and can't fill the next cadence, notify 7+ days ahead with two options: (a) wait for restock (no charge until ship), or (b) substitute a similar bag **from the same roaster. Never** substitute from Sorpresa/ Selezione/Offerta into a Roccia shipment.

6.1 Substitution on a subscription (Steve, 2026-08-20 - PARTLY OPEN)

A subscription is to a coffee, not to a lot. The lot is chosen by FIFO at fulfilment (§5.4).

Substitution is a last resort, not a convenience. If, at the moment a recurring order is generated, no compliant lot of the subscribed coffee exists (§5.4), we substitute rather than ship nothing.

How the substitute is chosen, in order: another coffee from the **same roaster**; failing that, the closest match on the customer's **saved taste profile** - roast level first, then flavour.

Caffeine is never substituted across. A decaf subscription is never filled with caffeinated coffee, or the reverse, under any circumstances. This is a hard constraint, not a preference.

Never at a higher price. If the substitute would cost more, the customer pays their normal rate; if less, they are charged the lower.

A substituted shipment counts as a shipment for subscriber and Founding Member benefits (§3, §4).

A substitution notice is transactional, not marketing. It concerns an order the customer is being charged for, so it is not subject to marketing preferences and cannot be opted out of. This matches the existing design note that transactional order emails are store-level rather than a customer toggle. Note that the account page's notification stub covers *marketing* preferences and Loop's delivery reminders; **neither gives the customer a choice about what ships**, which is a different question and is the one below.

OPEN - decide before the production build:

1. **Notify and allow a decline before shipping, or substitute and tell them with an easy return?** The first suits the brand's disclosure posture and delays the shipment while we wait; the second is operationally simpler and takes the choice away.
2. **Does the first-bag guarantee (§9) cover a substitution for an established subscriber?** Strictly it is a *first*-bag guarantee. The argument for covering it anyway is that we chose the coffee, not them, so we should carry the risk. It is a real cost either way.
3. **Should substitution be opt-out at signup?** A checkbox would settle it per customer rather than per incident. Steve left this open deliberately on 2026-08-20.

7. Collections / bundles — the BOM model

A collection (and any future bundle) is a **Bill-of-Materials SKU**, and every line of it is a real stocked SKU in its own right:

Line	Class	Count	Lot controlled?
The collection itself	`S`	1	No — it holds no stock and is assembled on order
Component coffees	`C`	N	**Yes**
Printed card	`I`	1	No
Presentation box	`K`	1	No

Classes are defined in the SKU Standard, *The type class*. **Lot control, roast-date capture and the freshness windows apply to class `C` only** — which is what a receiving procedure needs in order to know which lines require date capture at goods-in.

The BOM is transmitted per order, not stored by the 3PL. Each order emits its own pick-pack list naming all five kinds of line. We are not asking a fulfilment partner to hold and version our recipe, and we must not build anything that assumes they have. It also means the card-to-collection relationship travels with the order: the SKUs themselves share no visible pattern (SKU Standard, *The source segment*), so the BOM is the only place that link exists.

- **Browse facets are DERIVED from components, never hand-entered.** A collection's Region / Roast / Flavor / Caffeine values are the **union** of its components (Option A: positive to a filter when **any** component matches, per axis; AND across axes). Modelled in the POC via `component_handles` + `productFacets()`.

- **Availability is gated by components:** offered only while **all** components are in stock and within the freshness band **appropriate to that collection's shelf**; auto-pauses and returns automatically as stock rotates. For an ordinary collection that band is the fresh window; for an Offerta collection (below) it is the Offerta band. Reading the fresh window unconditionally would make an Offerta collection unpublishable by construction, with no visible reason why.
- **An aged collection is a NEW collection, deliberately curated — never an automatic transition.** A collection holds no stock and is assembled on order, so it cannot itself age; what ages are its components, and the gate above simply pauses it. Selling aged stock as a collection is therefore a decision to build one, with its own product code, priced on the aged factor in §2.2. This is what makes that factor live rather than vestigial, and it is a route for aged stock that is neither a single-bag markdown nor donation. It takes **no SKU suffix** — see the SKU Standard, *Collections, cards and boxes*.
- **Pre-enable gate: a collection is not enabled in Shopify until its coffees, its card and its box are all physically present at the 3PL.** The component gate above is continuous and automatic; this one is a single check at launch, and it is the one a person performs. Stock that has been ordered is not stock that has arrived, and a collection that goes live without its box ships as a bag in a mailer.
- **On-demand 3PL fulfilment:** on order, 3PL pulls components FIFO, assembles box + bags + card, QC's all roast dates against the band appropriate to that collection's shelf — the same band the availability gate reads, never `{freshness_window_days}` unconditionally — and ships in 1–2 business days.
- **Substitution matrix** (admin-defined per collection): if a component is within 7 days of its transition date or out of stock after order, 3PL may substitute a defined alternate from the same roaster; customer is told ("Same quality, same roaster, new terroir").
- **REQUIRED production capability:** an **admin-managed BOM builder** — create a collection by naming it and selecting component SKUs, facets/availability auto-derive, and each order emits a per-order pick-pack BOM to the 3PL. No developer, no code deploy. (See §11 tooling.)

---

8. Fulfilment & shipping

- **US-only at launch.** No international.
- **Free** on every Roccia **subscription** shipment, with **no minimum**.
- **One-time orders: free at $69+, flat $12.50 under $69.** Free-shipping progress bar in cart, **$69** threshold.
- **The free tier is contiguous U.S. only** - the lower 48 plus DC. Alaska, Hawaii, Puerto Rico and the territories get **calculated carrier rates**, because a free 1 kg order to those destinations loses money on every unit. Whether they instead get a *published* flat surcharge is open (§12.11).

- **The threshold is measured AFTER discounts.** The conservative reading, and the one both benchmarked competitors who state a basis use. A subscriber's 10% may not push a cart across the line at our expense. Interacts with the `MAX` no-stacking rule (§3).
- **Carriers:** USPS Ground Advantage under 1 lb; UPS Ground 1 lb+.
- **Transit:** East/Southeast 2–3 business days · Midwest/Mountain 3–4 · West Coast 4–5.

> **The $12.50 is an incentive spread, not a cost pass-through - do not "correct" it toward carrier cost.** Our USPS Ground Advantage commercial-base cost on a sub-1-lb order is materially lower, and that is deliberate: the number is the size of the gap the customer is being asked to close, on Steve's principle that *the difference between paying shipping and qualifying for free has to be compelling*. On a two-bag cart around $48, paying shipping costs $60.50 while adding a third bag reaches $72 and ships free - $11.50 more for $24 of coffee. At $9.50 the same choice costs $14.50 for the same bag and the nudge mostly disappears. $12.50 stays inside observed practice: above Miscela d'Oro's $10.95, below Eataly's $14.90.
>
> **Where $69 sits in the market.** A 21-seller benchmark of direct competitors, read from first-party live pages on 2026-08-22: cohort median threshold **$75**, coffee-DTC sub-median **$70**, median flat rate **$9.99** with a maximum of $14.90. $69 is level with Miscela d'Oro and just under Vergnano, so it stays the friendliest bar among Italian brand stores while still taking a genuine two-bag or kilo-plus cart to clear. $12.50 sits above Miscela d'Oro's $10.95 and below Eataly's $14.90. **This is the comparison set to re-run at the next review** (§8.3), not a one-time justification. Dataset: `Brand and Marketing\Market Research\Crema_Italia_Free_Shipping_Benchmark_v1.xlsx`.

8.3 Shipping rates are reviewed, not reacted to (Steve, 2026-08-24)

Carrier costs rise every year. The rates in §8 are a **lever we set**, not a cost we pass through, so they need a review cadence the way the freshness windows have a governance rule - otherwise margin erodes silently and the only signal is a bad year.

Review triggers. (1) **Annually, each autumn**, when USPS and UPS publish their General Rate Increase for the coming January. (2) On any **mid-year surcharge** that changes blended cost per order by more than a point. (3) Whenever the **3PL or its negotiated rates change**.

What the review measures. Not the carrier's headline percentage - **blended outbound cost per order**, split into the two bands that behave differently: orders **above** the threshold, where we absorb 100% of carrier cost, and orders **below** it, where we collect $12.50 against actual cost.

The exposure is weight and zone, not dollars, and that mismatch is the whole problem. The threshold is denominated in dollars; carrier cost is driven by weight and zone, and the two are not correlated. The worst cell is a cart that **just clears** the threshold while being heavy and going far: three 500 g bags at ~$72 to the West Coast crosses the one-pound boundary into UPS Ground at the furthest zone, and ships free. Model that cell explicitly at each review. A threshold that looks healthy on an average will still lose money on its own worst case.

The threshold is the preferred lever; the flat rate is not. Raising the **threshold** moves more orders into the paid band and lifts average order value, and customers read it as a number that varies between merchants. Raising the **flat rate** is the visible price of shipping and costs conversion on exactly the small first orders we most want to convert. So: **move the threshold first, and move the flat rate only to keep the spread compelling** (§8), never to chase carrier cost.

Both directions are allowed, unlike the freshness windows. §5.4's windows may only be shortened, because lengthening walks back a promise a customer bought under. Shipping rates carry no such asymmetry and may move either way - but **only prospectively**. An order already placed ships on the terms shown when it was placed.

Three structural options before repricing, in the order they should be considered. Repricing is the most visible lever and the least interesting one. 1. **Negotiated rates.** A 3PL's contracted carrier rates are routinely better than list, and often better than Shopify Shipping's built-in discount. This is a **3PL selection question and is not currently on the §12.9 qualifying list** - add it: *what are your negotiated USPS and UPS rates from a Tampa origin, by zone and weight band?* 2. **Packaging weight.** Dimensional weight and the one-pound USPS/UPS boundary are ours to influence. A mailer that keeps a two-bag order under a pound is worth more than a rate negotiation. 3. **Price, not shipping.** Free shipping is never free; it is amortised into the bag price. If carrier costs rise structurally, the honest answer is often a **pricing review** (§2.2), not a shipping-policy change. See §12.3 - outbound shipping is **not currently in the pricing model at all**.

8.1 Nothing inside a package shows a price (LOCKED — Steve, 2026-08-19)

Every package ships with a packing slip and nothing else. Prices, subtotals, discounts and totals never appear on any document inside any box, gift or not. The receipt is an **email entitlement**: Shopify's order confirmation goes to the purchaser's contact email and never travels physically.

Why it is a blanket rule rather than a gift-order exception. A conditional rule has to be executed correctly on every order by whoever is packing that day, and it fails silently — the customer only discovers it when a birthday present arrives with a price on it, by which point the damage is done and unrecoverable. A blanket rule cannot be forgotten, needs no flag to travel to the warehouse, and makes every order giftable by default.

What this obliges:

- The Shopify **packing slip template** (Settings → Shipping and delivery) must carry no monetary fields. It is a Liquid template we control; verify after any theme or settings change.
- **3PL selection question (§12.9):** *do you print our packing slip, or insert your own paperwork?* A 3PL that inserts its own pick list or invoice breaks this policy and no setting on our side prevents it. This is a qualifying question, not a preference.
- The order's **contact email stays the purchaser's**, always. A gift flow must never substitute the recipient's address there, or Shopify mails a priced confirmation to the person being surprised.

8.2 Gifting

- **Gifting is an order-level option, never per line.** A Shopify order carries exactly one shipping address, so a per-item gift flag would promise a split the platform cannot execute. Two recipients means two orders.

- **The control lives in the cart**, which we own; checkout takes no custom fields below Plus. It rides to the order as a cart attribute.
- **Gifting is never inferred.** A shipping address that differs from the billing address means the customer is shipping somewhere else — a vacation home, an office — and nothing more. Only an explicit opt-in makes an order a gift. Inferring it puts a gift card in a box someone bought for themselves.
- **A gift adds a printed card.** Same operational capability as the Sorpresa tasting card, so it is not new 3PL scope — but confirm insert format, character limit and per-insert cost.
- **Subscriptions cannot be gifted.** Any subscription line makes the whole order ungiftable: the cadence bills the giver's card indefinitely, and the order still has one address. A prepaid fixed-length gift subscription would be a **different product**, not a flag on this one, and we do not offer it. Nothing further is open here — this is the decision, not a deferral of one.

---

9. The Crema Italia Promise

Three lines, in this exact order, on a single editorial page linked from the footer: 1. Curated with love, never aggregated. 2. Love your first bag, or we send a different one - free. 3. Cancel your subscription anytime in a couple of clicks.

Below: first-bag satisfaction guarantee (standard shelves; Offerta is "as-is, defects only" per §5), damage replacement, the freshness window, and the no-waste pledge (Feeding Tampa Bay donations).

> The **exact customer-facing wording lives in the POC/theme** (the source of truth for *copy*); reproduce it from there and honor the no-em-dash rule (§10).

---

10. Strict exclusions (never appears on the site)

- **No grind options** anywhere — whole bean only (roasters seal valve bags at origin).
- **No claim** that we roast, blend, or process — we import and curate; the roasters roast.
- **No roast-day shipping-cadence claims** — pallets travel on a 6–10-week cadence.
- **No countdown timers, fake stock counts, or manufactured urgency** — real low-inventory and real freshness windows only.
- **No discount codes.** We never issue, publish, promote, or reference one (§3). Checkout's code field **is** visible — it cannot be hidden below Shopify Plus, and Plus is not justifiable for it (from $2,300/mo vs Advanced at $299; ~$24k/yr for one field — **declined 2026-07-25**). The field is inert in practice because no valid code exists. *This replaces v1.2's "No visible promo-code field at checkout," which asserted something we cannot build.*
- **No "have a promo code?" prompts, banners, or fields** anywhere **we** control — the storefront, emails, print, or packaging. The intent of the retired rule survives here: never train a customer to

go hunting for a code.

- **No sitewide percentage-off promotional sales.**

- **No emoji. No exclamation marks. No "amazing/best/you'll love it" copy.**
- **No em-dashes in customer-facing copy** (2026-07-13) — see Brand Standards / `CLAUDE.md` §6 for the replacement rule.

---

11. Data model & tooling (production)

Metafields (`crema_italia.*`): `roast_date` (Date), `lot_id` (Text), `eur_usd_rate` (Decimal), `landed_cost_usd` (Currency, locked at receipt), `markup_override` (Decimal, **optional** — blank = inherit the shelf/size matrix), `offerta_transition_date` (Date, computed - see §5), `current_shelf` (SingleSelect: Roccia|Sorpresa|Selezione|Offerta), `Referral_Gift_Allowed` (Boolean), plus taxonomy: `roast_level`, `flavor_profile`, `caffeine`, `shelf`, `region`, `roaster_handle`, `best_by_date`. Extend with a structured component-SKU BOM field on bundles (§7).

Retired fields - do not build, do not reintroduce. `days_to_offerta` (**RETIRED v1.12**: it was the freshness boundary stated a second time, and the two diverged, 45 against 90) and `peak_flavor_days` (**RETIRED v1.14**: advice we do not enforce, so it is brand copy, not policy). `best_by_date` is retained as a **computed, never-displayed** value (§5.4) - it is the roast date plus the window, so displaying it beside the window states one fact twice. The freshness values themselves are **not metafields at all**: they are a store-level declaration (§5.5).

The discount engine — REVISED v1.15 (2026-08-22) on measured platform behaviour. §3's no-codes, `MAX` policy is unchanged and still locked. What changed is **which system owns the rate**, because the v1.3 mechanism — one Function evaluating everything — was tested and does not survive contact.

Two owners, split by line type:

- **Subscription lines — the Loop selling plan owns the rate.** Founder 12% and subscriber 10% are **two selling plans**; a customer's rate is decided by which plan their contract is on, fixed at signup. Promoting someone to Founding Member is **migrating their contract** — a manual edit in the Loop admin, available on Loop's free tier, and bounded by the 222-slot cap in §4.
- **One-time lines — a Shopify Function owns the rate.** It evaluates `MAX` across the standing entitlement and every qualifying campaign, reading the customer's tags and metafields plus the cart.
- **The Function must not touch subscription lines.** Set `appliesOnSubscription: false`. A selling-plan adjustment is a **price change, not a discount**, so it never enters Shopify's combination contest and a Function discount **compounds on top of it** — measured at an effective **20.76%** where 12% was intended.

Why the single-evaluator model had to go. Discount Functions are **not re-run when recurring orders are created**; a Function discount is *snapshotted onto the subscription contract* at signup, carrying a `recurringCycleLimit` (`1` = first order only and the default, `N` = N cycles, `0` = indefinitely). A snapshot cannot re-evaluate, so a Function can never enforce a later tier change, a standing-rate change, or the §3.1 benefit lapse on an existing subscription. **Entitlement is contract state, not computed state.**

The one place this knowingly bends §3, accepted deliberately. With the rate on the plan, a campaign cannot out-rank it on a subscription line. Checked against the §3 table, only **one** campaign ever could: the **win-back 15%** at re-subscribe — every other campaign rate is at or below the 10% standing rate, so `MAX` is unaffected by them. That single gap is accepted for now.

Designed for, not built: a top-up to `MAX` on subscription lines. Shopify hands the Function both the plan-adjusted price and, on a subscription line only, `compareAtAmountPerQuantity` = the **pre-plan base price**. So the Function can discount *only the gap* between the plan's price and the best rate the customer qualifies for, restoring `MAX` exactly, with `recurringCycleLimit: 1` making the campaign genuinely one-time. Build this if and when win-back re-subscribes matter commercially; the design must not foreclose it.

Inputs the Function may rely on (verified, §12.7): customer `hasTags` / `hasAnyTag`, `numberOfOrders`, and custom-namespace customer **metafields** — the last needing no metafield definition and no access grant. The customer object is **null in the cart** and populated at **checkout**.

One operational rule that falls out of it: prefer a metafield to a tag for anything that must bite immediately. Tag writes propagate on a delay of minutes; metafield writes are readable on the next page load. A tag written by Flow or a Loop webhook moments before checkout may not be visible to the Function — so a resume that restores benefits, or a win-back window opening, belongs in a metafield.

Two consequences for reporting and the storefront, both measured: a selling-plan discount leaves **no trace on the order** — no discount line, no discount total, and the line's own "original" price is already reduced — so **Shopify's discount analytics will report zero discounts for the entire subscriber programme**, and **the theme must render the subscriber benefit itself** from the variant's base price against the selling-plan price.

Campaign eligibility remains a time-boxed tag or metafield, never a code (§3). A win-back email links to the store; the 15% is already attached to that customer.

Customer tags: `founding-member-NNN` (durable; set at signup, removed only on account closure — NO `founding_rate_forfeited`, retired per §4), plus subscription-state tags derived from Loop (actively-shipping / in-60-day-grace / lapsed) driving the benefit gate. Auto-managed by Shopify Flow / Loop webhooks. **Entitlement logic:** `benefits_on = (≥1 actively-shipping sub) OR (within 60-day grace)`; the *standing* rate `standing = founder ? 12% : 10%` when `benefits_on`, else 0%. The **applied** discount is `MAX(standing, any qualifying campaign discount)` per the no-stacking rule (§3) — never a sum.

Automation: Shopify Flow moves `current_shelf` → Offerta on `offerta_transition_date` and tags `active-roccia` on the $45+/month rule.

Account architecture split (LOCKED 2026-07-04): - **Loop** owns the active subscription's ship-to + payment + lifecycle notifications + the retention ("are you sure") prompt in its cancel flow. - **Native Shopify accounts** own the general address book + profile (name/email/password) + order history + buy-again + marketing consent. - **Entitlement is split by line type (v1.15):** the **Loop selling plan** carries the rate on subscription lines, and a **Shopify Function + customer tags/metafields** carries it on one-time lines. Tags and metafields are still maintained server-side by Flow and Loop webhooks, never by the theme, because a customer can cancel from an email link and never touch storefront UI.

The SKU price-maintenance engine (approach LOCKED 2026-07-13 — phased). Landed-cost × markup with the approval governance (§2.4) is **not a native Shopify feature**. Chosen path: - **At launch: spreadsheet-assisted** (compute in `Crema_Italia_Landed_Cost_Model_v1.xlsx`, enter prices manually in Shopify). Adequate for the small launch catalog on a 6–10-week lot cadence; governance = admin review at entry time. Zero build. - **Automation:** use **Shopify Flow** for what it does well — the Offerta **aging** transition (date-triggered shelf move + price recalc to `O[size]` + admin alert). Native, free. - **Later:** build a **lightweight custom app** (a proposed-price approve/hold/defer queue) **when volume justifies it** — trigger = SKU count / lot velocity high enough that manual pricing becomes a chore or error-prone. Back-office only; does **not** block or shape the storefront theme build.

---

12. Open decisions (must close before / at production build)

1. **~~Per-SKU markup override.~~ RESOLVED 2026-07-13** — matrix + optional per-SKU `markup_override` (blank = inherit); override is a deliberate exception and routes through admin approval. Rule now in §2.2; field in §11.
2. **~~The SKU price-maintenance tool mechanism.~~ RESOLVED 2026-07-13** — phased: spreadsheet + Shopify Flow (aging) at launch; lightweight custom app when volume justifies. Detail in §11.
3. **The markup matrix and the landed-cost model have never been reconciled, and they disagree.** The multipliers were specified, never run against real landed costs (the POC cart is mocked). **Pre-launch:** run real SKUs through both and make them agree.

First, a correction. v1.16 said outbound shipping *"appears nowhere in the pricing formula"* and that the matrix *"omits a cost line that scales with volume."* **That was wrong and is withdrawn.** `Crema_Italia_Landed_Cost_Model_v1.xlsx` carries outbound shipping in full: Inputs §D (`Outbound shipping $/order`, $6.50 / **$8.25** / $11.50, uncertain in the Monte Carlo), rolled into Cost Build §C `Per-order variable cost`, and into Pricing Scenarios as `Total unit cost = Landed $/bag + Per-order var`, from which MSRP is solved at a target margin. Nothing is omitted.

What is true is that §2.2 and that model are two independent pricing answers that have never been compared. §2.2 is `SKU_LAST_COST × Markup`, where `SKU_LAST_COST` is **inbound** landed cost only. That is not itself a defect - a markup multiplier is allowed to cover downstream costs implicitly. **The defect is that nobody checked whether it does.**

Computed 2026-08-24 at the model's own ML inputs (landed **$28.45/kg**; per-order variable cost **$14.85**, of which outbound shipping is **56%**), loading the per-order cost on a single-bag order as the model's Pricing Scenarios sheet does:

SKU	§2.2 price	Model unit cost	Gross margin
Sorpresa 100g (3.7×)	$10.53	$17.69	**−68%**
Roccia 250g (2.8×)	$19.92	$21.96	**−10%**
Selezione 250g (3.0×)	$21.34	$21.96	**−3%**
Roccia 500g (2.5×)	$35.57	$29.07	18%
Roccia 1kg (2.2×)	$62.60	$43.30	31%

Against a Pricing Scenarios sheet built around **50-70%**. Spreading the per-order cost across a two-bag order improves Roccia to 27% / 39% / 43% - still short of target. **And there are three different answers for the same 250g bag:** §2.2 says **$19.92**, the model's own "Current MSRP" input says **$17**, and the model's MSRP-at-60%-margin says **$54.90**.

The single largest variable is units per order, because outbound shipping and pick-pack are per-*order* costs being recovered by a per-*bag* price. So the pre-launch validation must first settle **the assumed basket size**, then ask whether the matrix delivers target margin at it. AOV is modelled at $42 against bag prices of $17-28, i.e. roughly 1.5-2.5 bags.

Two stale model inputs, found in the same pass. Inputs §E assumes **Shopify Basic** - plan $39, Payments **2.9%** - but the 2026-08-21 platform spike chose **Grow** ($79 billed annually, 2.7%), because Basic includes zero staff accounts. And the **$8.25** outbound figure was calibrated against the retired $8.50 flat-rate regime; it carries **no weight/zone split**, which is exactly the exposure §8.3 names, and the model credits **no shipping revenue**, so the $12.50 collected below the threshold is not offset anywhere. That last one is conservative; the first two are not.

This is also what decides whether a carrier increase is answered with a threshold move (§8.3) or a price move. 4. ~~**Pause-semantics reconciliation.**~~ **RESOLVED 2026-07-13** — benefits bound to ≥1 actively-shipping subscription; pause-all **and** cancel-all trigger a 60-day win-back grace, then lapse; reinstated on resume/re-subscribe. Founder status made **durable** (account-level, lost only on account closure), superseding the 2026-07-10 permanent-forfeiture model. Rules in §3.1 and §4. 5. **Deferred perks.** The month-12 "complimentary Selezione bag" annual perk was deferred to post-launch — confirmed deferred, tracked here so it isn't lost. 6. **Referral program — reward + capture TBD (2026-07-13).** No referral-capture mechanism has been built, and the former "free 100g bag" reward is void (no standalone 100g SKU — 100g exists only inside collections, §1). Both the reward form (e.g. a 250g bag, account credit) and the capture/tracking tooling are open; decide before any referral discount is enabled. The §3 table carries Referral as **TBD** until then. **Constraint added v1.3:** the chosen form must not require issuing a discount code (§3). 7. ~~**UNVERIFIED — can a Shopify discount Function read customer tags/metafields?**~~ **RESOLVED 2026-08-22 — YES, both.** Settled exactly as this item prescribed: a minimal Function deployed to the development store, reading one tag and taking 10% off. At checkout it read `hasAnyTag`, `hasTags`, `numberOfOrders` and a **custom-namespace customer metafield**, the metafield needing **no definition and no access grant**. Two caveats now carried in §11: the customer object is **null in the cart** and present only at **checkout**, and **tags propagate on a delay while metafields are immediate**, so anything that must take effect immediately belongs in a metafield. 8. ~~**UNVERIFIED — how does our discount Function coexist with Loop's selling-plan subscription discount?**~~ **ANSWERED 2026-08-22 — they compound, and it forced the v1.15 change to §11.** It was correctly called the highest-risk integration. Measured on a real subscription: a Function's 10% came off Loop's already-reduced price, billing an effective **20.76%**, and setting `combinesWith` to false on all three classes made no difference, because a selling-plan adjustment is a **price change, not a discount**, and never enters the combination contest. A second finding compounded it: **Functions are not re-run on recurring orders**, so the rate is snapshotted onto the contract at signup. The resolution is in §11: the plan owns subscription lines, the Function owns one-time lines, and `appliesOnSubscription: false` is the guard. 10. ~~**Where does the peak-flavour message live?**~~ **RESOLVED v1.14 (2026-08-21), opened v1.13.** Opened and closed within a day. The setting is **retired**; the message is merged into the whole-bean sentence (§5), reworded to count **from receiving** rather than from roast. The distinction that settled it is worth keeping: the freshness windows are **gates we enforce**, this is **advice about behaviour we do not control**, and only the former belong in settings.

9. **3PL selection — segregation, packing slip, and inserts.** No 3PL is selected. Three questions are **qualifying**, not preferences, and all should be asked before commercial terms.

(0) Can you pick from two age bands within one location, or would you require a bin transfer? Added 2026-08-21, **larger than the two below**, and **sharpened 2026-08-29 now that we have chosen an answer rather than three.** Because a SKU carries no shelf segment (SKU Standard, *There is no shelf segment*), an Offerta split creates a second Shopify product drawing on the same physical stock in the same bin, and FIFO would otherwise hand a full-price buyer the aged bag. Our preferred handling is **one bin, two pick conditions**, with the `-OFF` suffix on the pick line selecting the age band against the lot's roast date (SKU Standard, *The Offerta pick suffix*; alternatives recorded at `docs/production_build_spec.md` §13.9.2). **This is a qualifying question, not a preference** - some WMSs express an age-band allocation rule inside one location and some do not. If a candidate requires a physical transfer instead, **the per-unit transfer cost must be quoted before selection**, and the fallback is a marked bin, never per-bag stickering: the suffix is never printed on physical goods.

(a) *Do you print our packing slip, or insert your own paperwork?* A 3PL that inserts its own pick list or invoice breaks §8.1 and nothing on our side can prevent it. (b) *Can you insert a printed card, varied per order by an order attribute?* Format, character limit, per-insert cost, lead time. This is not new scope - Sorpresa collections already ship with a printed tasting card (§7), so a 3PL that cannot do it cannot fulfil Sorpresa at all. The gift card of §8.2 is the same capability asked once.

11. **Shipping to AK, HI, PR and the territories - published surcharge or calculated rates?** Opened v1.16 (2026-08-24). The free tier is contiguous-U.S. only (§8) and those destinations currently get **calculated carrier rates**, which is Steve's choice for now. The alternative is a *published* flat surcharge, the way Vergnano states a flat $32. Calculated rates never lose money but quote an unpredictable number at checkout; a published surcharge is friendlier to read and carries the risk. Low urgency, and it only bites once someone outside the lower 48 orders.
12. **The freshness declaration - four questions left by the 2026-08-23 decision.** The rule is settled in §5.5; these are implementation choices, and they are **build-gating** because getting them wrong means a rewrite rather than an edit.

(a) Metaobject or shop metafield? A singleton policy record could be either. The metaobject is proposed for the typed fields and the Admin editing surface; a shop metafield is simpler. Code's call.

(b) If a per-SKU freshness override is ever built, it must read this declaration as its default rather than introducing a second boundary. **That is precisely how `days_to_offerta` happened** - worth a comment in the code, not only here.

(c) Does `best_by_date` survive? §5.4 says it is never displayed and §11 retains it as a computed value. If nothing reads it, it is a field with no consumer, which is the defect Review A removed elsewhere in this project. Retire it or name its consumer.

(d) What exactly does the no-bare-literal lint cover? §5.5 specifies this source and the theme templates. Whether it also covers `docs/production_build_spec.md`, the legal policy sources and the OneDrive brief is open - each is a place a freshness number has already been written by hand. **Widened 2026-08-29:** §5.5 now also specifies two SKU rules for the same check, which makes the scope question sharper rather than looser - a SKU literal is far more likely to be written into a theme template than a freshness number is, and the check has to reach wherever that happens. **Note also that the lint does not exist yet**, so this decides the shape of something still to be written, not a change to something running.

13. **Pre-launch validation of every value in this Standard - the gate that cannot close early.** Opened 2026-08-24, and it is the reason for the callout at the top of this document. **After the live site is built and before it is made public**, walk the numbers deliberately rather than inheriting them: the pricing matrix and markups (§2.2) against real landed costs *including outbound shipping* (§12.3), the discount rates and the `MAX` interaction (§3), the shipping threshold and flat rate against real blended cost per order (§8.3), the freshness windows against real consumption (§5.4, which may only shorten them), the Founding Member cap (§4), and every minimum, maximum and cadence elsewhere in this file.

The timing is the whole point. Earlier is impossible - these cannot be validated against a store that does not exist. Later is too late - once public, they are commitments customers act on and we live with the outcome. This item therefore **cannot be closed at the production build** like the rest of §12; it closes in the window between build-complete and go-public, and it is the last gate before launch.

---

13. Reviews & social proof (Steve, 2026-08-20)

Numbered 13 rather than inserted next to the other customer-facing policy sections **because renumbering §10-§12 would falsify history** - those numbers are cited in other repo documents and, more importantly, inside `CLAUDE.md` §9 log entries, which are immutable records of what was true when written. Position in the file is worth less than keeping every existing citation true.

Reasoning, measurements and the rejected alternatives are recorded separately in the trust decision brief; this section states only what is now true.

13.1 Who may submit a review - purchase-gated only

Reviews may be submitted **only by a verified purchaser of that product**, through a per-order link delivered by email after fulfilment. The public storefront review form is **disabled**. There is no route by which a person who has not bought the product can leave a review of it.

Note the distinction that decided this: an account is free and anyone can create one, so a login gate proves nothing. **The order is the trust anchor**, not the sign-in. The platform's review schema agrees - it carries an order reference and a verification status alongside the author.

13.2 Anonymity - identity retained, display optional

A reviewer may choose the name shown, including a first name only or a neutral mark such as *Verified buyer*. The **customer identity is retained internally regardless**, because it is the join key to that customer's saved taste profile (§13.6). Privacy on the page; the link underneath.

13.3 What we publish, and what we do not

Everything except abusive content is published, including critical and negative reviews. A product page showing only five-star reviews reads as curated, which destroys the trust it is meant to build.

Abusive, for the avoidance of a judgment call made differently by whoever opens the queue each day, means: personal attacks on a named individual; obscenity; content about a person rather than the coffee; content that is not about the product at all; and anything unlawful. **A complaint about the coffee is not abuse and is published.** Service and delivery complaints belong in support, and are redirected rather than published.

Merchant replies are permitted and encouraged on critical reviews. A considered reply is itself a trust asset and suits the editorial voice better than a defensive one.

13.4 No photograph reviews

Customer photographs are **not collected and not displayed**. They are not evidence of satisfaction in either direction, and on a store whose own photography is governed by Brand Standards §3.5 (natural light, low saturation, narrow depth of field) an uncurated stream of phone snapshots would become the store's visual identity by default.

13.5 How a rating is displayed

The rating is rendered through **our own discreet control**, designed to sit inside the brand rather than a review vendor's widget furniture. It appears on the product page whether or not ratings exist, and links to a dedicated review-detail view.

- **With ratings:** the control shows the aggregate and the count, quietly, and links to detail.
- **With none:** it says so plainly and points a purchaser at the route in. Approved copy: *"No ratings exist for this product. Have you purchased this product? If yes, check your email and be the first to submit a rating."*
- **Machine-readable markup** (`aggregateRating`) is emitted **only when at least one real review exists**. Never for zero. See `docs/production_build_spec.md` §9.2.

13.5.1 Where the control may appear (Steve, 2026-08-20)

Only on the detail view of a purchasable product. Never on a roaster profile, a person page, a collection landing surface, or any editorial page. A roaster is not rated; a coffee is. The guard is explicit because the control is otherwise easy to switch on somewhere it does not belong.

Never on a product card in a grid, while below the catalogue coverage floor. This is §13.6's principle applied one level up: that section keeps a per-product signal silent below its *sample* floor; this keeps the card-level mark silent below a *coverage* floor. Three reasons, and the first is the one that decides it:

1. **A grid is a comparison device, and this rating is not comparable.** The premise of this whole section is that an average measures *did this match my palate*, not *is this good*. Set those averages side by side on a shelf page and the store has built the exact comparison it says is invalid - a shopper picks 4.6 over 4.2 and is choosing on noise. The same number on a detail page is information about one coffee; in a grid it becomes a ranking.
2. **The empty state does not survive repetition.** One "Not yet rated" is an invitation. Thirteen down a shelf page is a wall of nulls announcing that nothing has been reviewed.
3. **Uneven coverage distorts merchandising.** With a few products rated and most not, the rated ones read as endorsed and the rest as ignored - an artefact of which sold first, not of quality, and it works against every newly signed roaster.

The floor: the card-level mark stays off until a clear majority of the active catalogue has cleared the per-product floor of §13.6, at which point reason 3 has resolved itself. Above the floor, the mark may appear **only on cards that have a rating** - a null is never rendered in a grid under any circumstances.

13.5.2 Bottega is its own rating context (Steve, 2026-08-20 - closes the v1.9 open item)

Bottega is rated, and is not exempt. Reason 1 above is an argument about *coffee*, where the palate is the variable. It does not apply to equipment: a grinder that will not hold its setting is bad for everyone, so a global average on a Bottega item is valid in a way it is not on a roast, and is arguably more useful - someone spending real money on a grinder wants exactly that number.

This is not an inconsistency. Bottega is **already** an exception throughout the store, and says so to the customer: *"Bottega items are never subscriber-discounted and are not part of the four coffee shelves."* Treating it as its own rating context is consistent with how it is treated everywhere else.

Two consequences, both binding:

- **Bottega never shows a reorder rate.** Nobody rebuys a grinder, so the figure would sit near zero, mean nothing, and read as damning. It is excluded **by shelf**, not left for the §13.6 sample floor to catch - a floor would eventually pass it and publish a meaningless number.
- **Bottega never gets the palate-matched layer** (§13.6). There is no palate involved.

And one consequence for later: **Bottega is the one shelf where a card-level rating could be defended**, because §13.5.1's first and deciding objection - that a grid invites a comparison this rating cannot support - does not hold for equipment. It stays off cards for now with the rest of the catalogue; if the coverage floor is ever revisited, Bottega may be revisited on its own terms.

13.6 Reorder rate and palate-matched feedback - floors, and silence below them

Both are permitted and both are **gated on sample size**. Each display carries a minimum-*n* floor, and **below the floor the store shows nothing at all**. Showing nothing is honest; showing a percentage derived from a handful of orders is not, and it is indistinguishable to the reader from a number that means something.

This is why neither is a launch feature: reorder rate is undefined until customers have had the chance to reorder, and palate-matched feedback divides the sample across taste segments, so it needs roughly an order of magnitude more data than a single average does.

13.7 Never fabricate

No rating, review, count, reorder rate, or percentage may ever be displayed on the live store unless it derives from real customer activity. This includes seeded, sample, illustrative and "representative" data of every kind.

The POC is explicitly exempt and always has been - its entire catalogue is invented - but fixture review data must be **named so that it cannot be mistaken for real or shipped by accident**, in the same way `ci-temp-*` marks the placeholder photography. The same rule governs photography: no placeholder image ships in the real build.

Store Operating Standards v1.18 · 2026-08-29 · Source of truth: `docs/standards/store-operating-standards.md`. Renders (PDF for humans / Cowork) are read-only snapshots stamped with this version — edit the source, not the render.